

Digital Image Processing (4th Ed.) – Chapter 10 Solutions

Problems 10.9, 10.16, 10.17, 10.21, 10.28, 10.29, 10.36, 10.38, 10.39

Problem 10.9 ★

Question: Suppose we used the edge models in the given image (instead of the ramp in Fig. 10.10). Sketch the gradient and Laplacian of each profile.

The image in the problem shows two edge model profiles:

- **Profile (a):** A step edge – the intensity drops from a high plateau to zero at one side (a rectangular/plateau shape between two step transitions).
- **Profile (b):** A roof edge – a symmetric triangular peak (dark background → linear rise → peak → linear fall → dark background).

Profile (a): Step Edge (Plateau / Double Step)

The intensity profile is:

$$f(x) = \begin{cases} 0 & \text{for } x < x_1 \\ A & \text{for } x_1 \leq x \leq x_2 \\ 0 & \text{for } x > x_2 \end{cases}$$

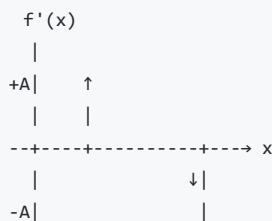
This is a step UP at x_1 and a step DOWN at x_2 .

Gradient (first derivative):

At an ideal step, the first derivative is a Dirac impulse (in discrete terms, a large finite spike):

$$f'(x) = \begin{cases} +A \cdot \delta(x - x_1) & \text{[positive spike at left edge]} \\ -A \cdot \delta(x - x_2) & \text{[negative spike at right edge]} \\ 0 & \text{[everywhere else]} \end{cases}$$

Sketch:



The gradient magnitude image shows two sharp bright peaks at each transition.

Laplacian (second derivative):

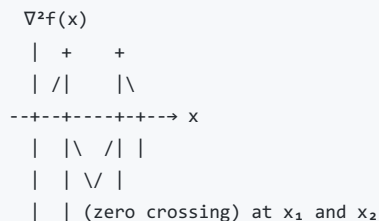
The second derivative of a step is the derivative of the delta function (a doublet/dipole):

- At x_1 (rising step): $+\infty$ spike immediately followed by $-\infty$ spike (in continuous; in discrete: a positive value at x_1-1/x_1 and negative at x_1/x_1+1)
- At x_2 (falling step): $-\infty$ spike followed by $+\infty$ spike

In the discrete, practical sense using a 1-D second-difference kernel $[1, -2, 1]$:

$$\nabla^2 f = \begin{cases} \text{positive spike then negative spike at } x_1 \\ \text{negative spike then positive spike at } x_2 \\ \text{zero between and outside the transitions} \end{cases}$$

Sketch:



Note the **zero crossing** between the two values at each edge – this is what the Marr-Hildreth detector locates as the edge center.

Profile (b): Roof Edge

The intensity profile is:

$$f(x) = \begin{cases} 0 & \text{for } x < x_1 \text{ (dark background, left)} \\ \text{slope up} & \text{for } x_1 \leq x \leq x_{\text{peak}} \text{ (linearly rising left side)} \\ \text{slope down} & \text{for } x_{\text{peak}} \leq x \leq x_2 \text{ (linearly falling right side)} \\ 0 & \text{for } x > x_2 \text{ (dark background, right)} \end{cases}$$

Gradient (first derivative):

On the linearly rising left side: $f'(x)$ = constant positive value $c > 0$. At the peak: f' transitions from $+c$ to $-c$ (sign change). On the linearly falling right side: $f'(x)$ = constant negative value $-c$. In flat background: $f'(x) = 0$.

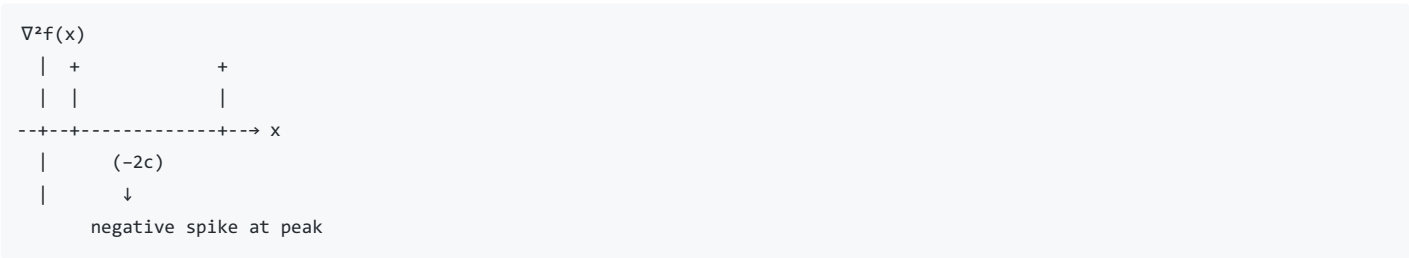


The gradient magnitude shows two constant positive bands on each slope (both sides read as +c in magnitude).

Laplacian (second derivative):

The second derivative of a piecewise linear function is zero on flat/constant-slope regions and produces impulses wherever the slope changes:

- At x_1 (slope changes from 0 to $+c$): **positive impulse** (slope increases)
- At x_{peak} (slope changes from $+c$ to $-c$): **strong negative impulse** (slope decreases sharply – this is the zero crossing location)
- At x_2 (slope changes from $-c$ to 0): **positive impulse** (slope increases again)



The zero crossing of $\nabla^2 f$ at the peak of the roof identifies the center of the line/roof edge – this is the edge point location.

Key Comparison with the Ramp (Fig. 10.10):

Edge Type	Gradient	Laplacian	Zero Crossing
Ramp	Constant positive plateau during ramp	+spike at start, -spike at end of ramp	At center of ramp
Step	Delta impulse at transition	Doublet (bipolar) at each transition	At the step location
Roof	Constant positive/negative on each slope	+spike at base, -spike at peak	At peak of roof

Problem 10.16

Reference: Eq. (10-29): The Laplacian of a Gaussian (LoG): $\nabla^2 G(x,y) = \frac{x^2+y^2-2\sigma^2}{\sigma^4}e^{-\frac{x^2+y^2}{2\sigma^2}}$

Fig. 10.4(a): The 3×3 Laplacian kernel:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

whose coefficients sum to zero.

Part (a) ★ — Show that the average value of $\nabla^2 G(x,y)$ is zero.

The average value of a function $h(x,y)$ over an infinite domain is proportional to its Fourier transform evaluated at the origin: $\text{avg}\{h\} \propto H(0,0) = \mathcal{F}\{h\}|_{u=0,v=0}$

From Fourier transform theory, the Laplacian operator in the frequency domain corresponds to multiplication by $-4\pi^2(u^2+v^2)$: $\mathcal{F}\{\nabla^2 G(x,y)\} = -4\pi^2(u^2+v^2) \mathcal{F}\{G(u,v)\}$ where $\mathcal{F}\{G(u,v)\}$ is the Fourier transform of the Gaussian (itself a Gaussian).

Evaluating at $(u=0, v=0)$: $\mathcal{F}\{\nabla^2 G\}|_{(0,0)} = -4\pi^2(0+0) \mathcal{F}\{G(0,0)\} = 0$

Therefore: **$\text{avg}\{\nabla^2 G(x,y)\} = 0$** ■

Alternative (direct integration): By Green's identity / the divergence theorem: $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \nabla^2 G \, dx \, dy = \oint_{\partial\Omega} \nabla G \cdot \hat{n} \, ds \rightarrow 0$ as $|\Omega| \rightarrow \mathbb{R}^2$ since G and ∇G both decay to zero exponentially at infinity.

Part (b) — Show that the average of any image convolved with $\nabla^2 G$ is also zero.

Let the convolved image be: $h(x,y) = [\nabla^2 G(x,y)] * f(x,y)$

By the Convolution Theorem: $H(u,v) = \mathcal{F}\{\nabla^2 G\}(u,v) \mathcal{F}\{f\}(u,v)$

The average value of h is: $\text{avg}\{h\} \propto H(0,0) = \mathcal{F}\{\nabla^2 G\}(0,0) \mathcal{F}\{f\}(0,0) = 0 \mathcal{F}\{f\}(0,0) = 0$

since we showed in part (a) that $\mathcal{F}\{\nabla^2 G\}(0,0) = 0$.

Therefore, **for any image $f(x,y)$, the average value of $[\nabla^2 G * f]$ is zero.** ■

Part (c) — Using Fig. 10.4(a) to approximate LoG, then convolving with any image.

The kernel in Fig. 10.4(a) is: $w = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

Sum of coefficients = $1+1+1-8+1+1+1 = 0$.

From Problem 3.32 (referenced in the hint): when a kernel whose coefficients sum to zero is convolved with any image, **the sum of all values in the resulting image is zero.**

This is because convolution can be written as: $\sum_{x,y} h(x,y) = \left(\sum_{s,t} w(s,t) \right) \left(\sum_{x,y} f(x,y) \right) = 0 \cdot \text{anything} = 0$

Conclusion: If we use the kernel in Fig. 10.4(a) to approximate the Laplacian of a Gaussian and convolve it with any image, **the resulting image will always have pixel values that sum to zero.** This means the overall "DC component" or average intensity of the output is always zero, regardless of the input image. This property is essential — it ensures the LoG kernel gives zero response in regions of constant intensity (a necessary requirement for an edge detector).

Problem 10.17

Reference: Fig. 10.22(c) shows the zero crossings of the Marr-Hildreth algorithm (Steps 1 and 2) applied with a threshold of zero, producing a result where **all edges form closed contours** — the so-called "spaghetti effect."

Part (a) — Why do the edges form closed contours in Fig. 10.22(c)?

The key reason is that the threshold used is **zero**.

The LoG-filtered image $g(x,y) = [\nabla^2 G] * f$ is a continuous real-valued function. By the Intermediate Value Theorem (in 2D topology), the **zero-level set** of any continuous 2D function — the set of points where $g = 0$ — forms **closed curves** in $\mathbb{R}^2$. This is a topological property: zero crossings of a 2D function always partition the plane into alternating positive and negative regions, and the boundaries between those regions are closed contours.

In the discrete domain with threshold $T = 0$, a pixel p is marked as a zero-crossing edge if **any** pair of its opposing neighbors have different signs. Because the LoG image oscillates around zero everywhere (even in featureless regions, due to noise and quantization), practically every pixel will have at least one sign change in its 3×3 neighborhood when $T = 0$. This causes edges to be detected everywhere, forming continuous closed loops throughout the image.

Summary: With $T = 0$, zero crossings exist everywhere because the LoG function has alternating signs across the entire image. Since they are everywhere and connected, they must form closed loops.

Part (b) ★ — Does the zero-crossing method always result in closed contours?

No — not always, specifically when a positive threshold $T > 0$ is used.

With $T = 0$: Yes, closed contours always result (as explained above). This is the topological argument.

With $T > 0$: The zero-crossing condition becomes stricter: a pixel p is marked as an edge only if the **absolute difference** between opposing neighbors with different signs exceeds T . In regions where the LoG signal is very weak (e.g., gradual, smooth intensity changes), the sign change is present but the magnitude is small. Such

pixels are **suppressed** by the threshold, breaking the contours.

This is exactly what Fig. 10.22(d) shows: with a positive threshold (~4% of maximum LoG value), the closed loops break into open edge segments, and only the strongest, most significant edges are retained. The "spaghetti effect" disappears.

Conclusion: The zero-crossing method produces closed contours **only when T = 0**. With T > 0, open edges are possible and desirable.

Problem 10.21

Context: The Marr-Hildreth Algorithm:

- **Step 1:** Convolve image $f(x,y)$ with an $n \times n$ Gaussian kernel $G(x,y)$ sampled from Eq. (10-27)
- **Step 2:** Compute Laplacian using the 3×3 kernel in Fig. 10.4(a)

The combined operation: $g(x,y) = \nabla^2[G(x,y) * f(x,y)]$

Part (a) – Show Steps 1 and 2 can be implemented with four 1-D convolutions.

Step 1 (Gaussian smoothing) – 2 convolutions:

The 2-D Gaussian is separable: $G(x,y) = G_x(x) \cdot G_y(y) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2} \cdot \frac{1}{\sqrt{2\pi}\sigma} e^{-y^2/2\sigma^2}$

By separability (Problem 10.20a), the 2-D convolution $G * f$ reduces to two 1-D convolutions:

1. Convolve each **row** of f with the 1-D kernel $g(x)$ (a sampled Gaussian vector) $\rightarrow$ intermediate image f_{row}
2. Convolve each **column** of f_{row} with $g(y) \rightarrow$ smoothed image $f_s = G * f$

This requires **2 one-dimensional convolutions**.

Step 2 (Laplacian) – 2 convolutions:

The Laplacian operator decomposes as: $\nabla^2 f_s = \frac{\partial^2 f_s}{\partial x^2} + \frac{\partial^2 f_s}{\partial y^2}$

Each second-order partial derivative can be computed with a 1-D kernel $[1, -2, 1]$ (or its transpose):

3. Convolve each **row** of f_s with $[1, -2, 1] \rightarrow \partial^2 f_s / \partial x^2$
4. Convolve each **column** of f_s with $[1, -2, 1]^T \rightarrow \partial^2 f_s / \partial y^2$

Sum the two results to get $\nabla^2 f_s$.

This requires **2 more one-dimensional convolutions**.

Total: 4 one-dimensional convolutions implement both steps. ■

Part (b) – Derive the computational advantage.

Assumptions: G sampled into an $n \times n$ kernel; image f is $M \times N$ pixels.

Direct 2-D convolution (Step 1 only, the expensive part): Each output pixel requires n^2 multiplications (one per kernel element). Total multiplications = $M \cdot N \cdot n^2$

1-D decomposition for Step 1:

- Row pass: $M \cdot N \cdot n$ multiplications
- Column pass: $M \cdot N \cdot n$ multiplications
- Total = $2 \cdot M \cdot N \cdot n$

Computational Advantage (for Step 1): $\text{Advantage}_{\text{Step 1}} = \frac{M \cdot N \cdot n^2}{2 \cdot M \cdot N \cdot n} = \frac{n}{2}$

Including Step 2 (Laplacian with $[1, -2, 1]$ kernel, size 3):

Method	Multiplications
Direct 2-D ($n \times n$ Gaussian + 3×3 Laplacian)	$MN(n^2 + 9)$
Four 1-D convolutions	$MN(2n + 6)$

$\boxed{\text{Computational Advantage}} = \frac{n^2 + 9}{2n + 6}$

For large n (typical σ requires $n \approx 6\sigma$, so n can be 25–50): $\text{Advantage} \approx \frac{n^2}{2n} = \frac{n}{2}$

For example, with $n = 25$: advantage $\approx 12.5\times$ (the 1-D approach is ~ 12 times faster).

Problem 10.28

Scenario: Binary images of bubble chamber events. Tracks are 1-pixel thick straight lines at specific angles: $\pm 25^\circ, \pm 50^\circ, \pm 75^\circ$ off horizontal, with $\pm 5^\circ$ tolerance. Valid track: ≥ 100 pixels long, ≤ 3 gaps each ≤ 10 pixels. Must differentiate tracks with same direction but different origins.

Proposed Segmentation Approach Using the Hough Transform

Step 1 – Preprocessing (stated as given): Images are already binary with 1-pixel-thick tracks. If needed, apply morphological thinning to ensure exactly 1-pixel width.

Step 2 – Apply the Hough Transform in (r, θ) space:

Use the normal parameterization (Eq. 10-44): $x \cos \theta + y \sin \theta = r$

Set up the accumulator array $A(r, \theta)$ with:

- θ range: $-90^\circ \leq \theta \leq 90^\circ$, resolution $\Delta\theta = 1^\circ$
- r range: $-D \leq r \leq D$ (where D = diagonal of image), resolution $\Delta r = 1$ pixel

For every foreground (non-background) pixel (x_k, y_k) : for each θ in the allowed ranges (see Step 3), compute $r = x_k \cos \theta + y_k \sin \theta$, and increment $A(r, \theta)$.

Step 3 – Restrict to Relevant Angle Bands:

Only examine accumulator cells within these six angle bands:

Target Angle	Tolerance	θ Range
+25°	±5°	[20°, 30°]
-25°	±5°	[-30°, -20°]
+50°	±5°	[45°, 55°]
-50°	±5°	[-55°, -45°]
+75°	±5°	[70°, 80°]
-75°	±5°	[-80°, -70°]

Step 4 – Peak Detection:

Find local maxima in $A(r, \theta)$ within each angle band. Each peak (r^*, θ^*) corresponds to a candidate track line. Set a minimum threshold $K_{\min}$ on the accumulator value (e.g., $K_{\min} \approx 90$, slightly less than the 100-pixel minimum track length, to account for gaps).

Step 5 – Track Validation:

For each detected peak (r^*, θ^*) :

- (a) **Map back to image plane:** Identify all pixels (x_k, y_k) that contributed to cell (r^*, θ^*) , i.e., pixels satisfying $|x_k \cos \theta^* + y_k \sin \theta^* - r^*| < \text{tolerance}$.
- (b) **Length check:** Count total matched pixels. Accept only if count ≥ 100 .
- (c) **Gap check:** Sort matched pixels along the track direction. Measure gaps between consecutive pixels. Accept only if: number of gaps ≤ 3 AND each gap length ≤ 10 pixels.
- (d) **Origin differentiation:** Two tracks with the same angle θ but different origins correspond to **different values of r** in the accumulator and thus produce **separate peaks** in the (r, θ) space. The Hough transform inherently differentiates them – no extra processing needed.

Step 6 – Collision Point Detection (optional):

Tracks emanating from a single collision point converge at one (x, y) in image space. Compute pairwise intersections of accepted track lines: if multiple tracks share a common intersection point within a small tolerance, mark it as a collision vertex.

Summary: The Hough transform is ideal here because it:

1. Detects collinear points (complete tracks) even in the presence of gaps
2. Automatically separates parallel tracks by their r value (origin)
3. Can be restricted to specific angle ranges for efficiency

Problem 10.29 ★

Original algorithm (pixel-based): Iteratively threshold the image, compute group means over all pixels, update T .

Restated Algorithm Using the Histogram

Let $p_i = n_i / (MN)$ be the normalized histogram (n_i = number of pixels with intensity i , $i = 0, 1, \dots, L-1$). The histogram is computed **once**.

Algorithm:

1. Compute the normalized histogram $\{p_i, i = 0, 1, \dots, L-1\}$ from $f(x,y)$. This is done once: $O(MN)$ time.
2. Select initial threshold T (e.g., the global mean: $T_0 = \sum_i i \cdot p_i$).
3. **Using T , partition histogram bins** into two groups:

- C_1 : intensity levels $i \leq \lfloor T \rfloor$ (corresponding to pixels with values $\leq T$)
- C_2 : intensity levels $i > \lfloor T \rfloor$ (corresponding to pixels with values $> T$)

4. Compute class probabilities: $P_1 = \sum_{i=0}^{\lfloor T \rfloor} p_i$, $P_2 = 1 - P_1$
5. Compute class means from the histogram: $m_1 = \frac{1}{P_1} \sum_{i=0}^{\lfloor T \rfloor} i \cdot p_i$, $m_2 = \frac{1}{P_2} \sum_{i=\lfloor T \rfloor+1}^{L-1} i \cdot p_i$
6. Update threshold: $T_{\text{new}} = \frac{m_1 + m_2}{2}$
7. Repeat Steps 3–6 until $|T_{\text{new}} - T_{\text{old}}| < \Delta T$.

Advantage: The histogram (size $L = 256$ for 8-bit images) is computed once in $O(MN)$. Each iteration only requires $O(L)$ operations scanning the histogram — compared to $O(MN)$ per iteration in the pixel-based version. Since $L \ll MN$ for large images, this is significantly more efficient.

Problem 10.36

Statement: Show that a maximum value for Eq. (10-63) always exists for k in $0 \leq k \leq L-1$.

Eq. (10-63): $\sigma_B^2(k) = \max_{0 \leq k \leq L-1} \sigma_B^2(k)$ where $\sigma_B^2(k) = \frac{[m_G P_1(k) - m(k)]^2}{P_1(k)[1 - P_1(k)]}$, valid for $0 < P_1(k) < 1$.

Proof

Step 1 — The domain is finite and nonempty.

The domain is the set of integer values $k \in \{0, 1, 2, \dots, L-1\}$ restricted to those k for which $0 < P_1(k) < 1$.

For this set to be nonempty, the image must have at least two distinct intensity levels (otherwise all pixels have the same value and thresholding is meaningless — assume this is so). When at least two distinct levels exist, there exists at least one k such that $0 < P_1(k) < 1$.

Step 2 — $\sigma_B^2(k)$ is real-valued and bounded.

$\sigma_B^2(k)$ is defined for all valid k as a ratio of a squared quantity (numerator ≥ 0) divided by a positive quantity $P_1(k)[1 - P_1(k)] \in (0, 1/4]$. It is finite and non-negative for all valid k .

Step 3 — A finite nonempty set of real numbers always has a maximum.

The set $\{\sigma_B^2(k) : 0 < P_1(k) < 1, k \text{ integer}\}$ is a finite collection of real numbers (since k ranges over a finite set). Any finite nonempty set of real numbers has a maximum element (the largest value in the set). Therefore, a k^* achieving $\max \sigma_B^2(k)$ always exists. ■

Note: The maximum need not be unique — if multiple values of k achieve the same maximum σ_B^2 , the convention (stated in the text) is to average those k values to obtain k^* .

Problem 10.38

Context: Image $f(x,y)$ has intensities in $[0, 1]$. Threshold T segments f successfully.

Part (a) ★ — Show $T' = 1 - T$ segments the negative of $f(x,y)$.

Definitions:

- Original segmentation with T : pixels with $f > T \rightarrow$ object, pixels with $f \leq T \rightarrow$ background.
- Negative image: $f'(x,y) = 1 - f(x,y)$ (Section 3.2, negative transformation: maps $[0,1]$ to $[1,0]$).
- Proposed new threshold: $T' = 1 - T$.

Proof:

A pixel (x,y) satisfies $f'(x,y) > T'$: $1 - f(x,y) > 1 - T \Leftrightarrow -f(x,y) > -T \Leftrightarrow f(x,y) < T$

So, pixels classified as "object" by T' in f' are exactly those with $f(x,y) < T$ — these are the **background pixels** in the original f .

Similarly, pixels with $f'(x,y) \leq T'$ satisfy $f(x,y) \geq T$ — these are the **object pixels** in the original f .

The segmentation boundary (the set of pixels at the transition $f = T$) maps exactly to $f' = 1 - T = T'$. Therefore, **$T' = 1 - T$ correctly segments the negative image f' into the same two regions** as T segments f (labels are swapped: original objects become "low intensity" in f' , but the spatial partition is identical). ■

Part (b) — Conditions for arbitrary intensity transformation to preserve segmentability.

Let $T_{\text{transform}}$ be an arbitrary intensity transformation function (different from the threshold), so that $g(x,y) = T_{\text{transform}}(f(x,y))$.

Required conditions for segmentability to be preserved:

Condition: The transformation $T_{\text{transform}}$ must be **strictly monotone** (either strictly increasing or strictly decreasing) on the range of f .

- **Strictly increasing:** The ordering of pixel intensities is preserved: $f(x,y) > f(x',y') \iff g(x,y) > g(x',y')$. The threshold on g that produces the same segmentation as T on f is: $T' = T \circ T^{-1}$
- **Strictly decreasing** (like the negative): The ordering reverses: $f(x,y) > f(x',y') \iff g(x,y) < g(x',y')$. The spatial regions are preserved but roles (object/background) are swapped. The new threshold is again: $T' = T \circ T^{-1}$

Why strict monotonicity is necessary: If T -transform is not monotone (e.g., non-invertible), then two different intensity levels in f could map to the same value in g , or the relative ordering of intensities changes in a non-systematic way. This means there is no single threshold on g that can recover the same spatial segmentation as T on f .

Summary:

- Condition: T -transform must be **strictly monotone** (increasing or decreasing).
- New threshold value: $T' = T \circ T^{-1}(T)$ (the image of the original threshold under the transformation).

For the specific case in part (a): $T\text{-transform}(z) = 1 - z$ is strictly decreasing, and $T\text{-transform}(T) = 1 - T = T'$. ✓

Problem 10.39

Given:

- Object pixels: mean intensity $\mu_1 = 170$ (on $[0, 255]$)
- Background pixels: mean intensity $\mu_2 = 60$
- Noise: Gaussian, zero mean, $\sigma_n = 10$ intensity levels
- Required: segmentation rate $\geq 90\%$

Analysis of Class Separation

After adding Gaussian noise, the intensity distribution of each class becomes:

- **Object pixels:** approximately $N(\mu_1, \sigma_n^2) = N(170, 100)$ — Gaussian centered at 170
- **Background pixels:** approximately $N(\mu_2, \sigma_n^2) = N(60, 100)$ — Gaussian centered at 60

The **separation** between the means is: $|\mu_1 - \mu_2| = |170 - 60| = 110$ intensity levels $= \frac{110}{\sigma_n} = 11\sigma_n$

This is an **enormous separation** of 11 standard deviations. From the $\pm 3\sigma$ rule: 99.7% of each class's pixels fall within $\pm 3\sigma = \pm 30$ of their respective means:

- Background: 99.7% of background pixels lie in $[30, 90]$
- Objects: 99.7% of object pixels lie in $[140, 200]$
- These ranges **do not overlap** — the gap between them is $[90, 140]$.

Proposed Thresholding Method

Method: Basic global thresholding (or Otsu's method) with initial $T = (\mu_1 + \mu_2)/2$

Algorithm:

1. Set initial threshold $T_0 = (170 + 60)/2 = 115$
2. Apply the basic global thresholding algorithm (or compute histogram and use Otsu's method)
3. The algorithm converges quickly (likely in 1–2 iterations) to $T \approx 115$
4. **Threshold the image:** pixels with intensity $> 115 \rightarrow$ object; $\leq 115 \rightarrow$ background

Verification of $\geq 90\%$ Correct Segmentation Rate

For threshold $T = 115$:

Misclassification rate for background pixels (background pixel classified as object): $P(\text{error}|\text{bg}) = P(X > 115 \mid X \sim N(60, 10^2)) = P(\text{left}(Z > \frac{115-60}{10})\text{right}) = P(Z > 5.5) \approx 0.0000019 < 0.0002\%$

Misclassification rate for object pixels (object pixel classified as background): $P(\text{error}|\text{obj}) = P(X < 115 \mid X \sim N(170, 10^2)) = P(\text{left}(Z < \frac{115-170}{10})\text{right}) = P(Z < -5.5) \approx 0.0000019 < 0.0002\%$

Overall correct segmentation rate $\approx 99.9998\% \gg 90\%$ ✓

Even with the more conservative $\pm 3\sigma$ threshold:

- Any $T \in [90, 140]$ achieves error $< 0.13\%$ on each class
- Correct rate $> 99.7\% \gg 90\%$ ✓

Why This Works

The Gaussian noise with $\sigma = 10$ barely affects the outcome because the two classes are separated by 11σ . The basic global thresholding algorithm is well-suited here because the image histogram is strongly **bimodal** with a deep valley around 115, making threshold detection trivial and robust.

Property	Value
Class separation	110 = 11 σ
Optimal threshold T	115
Valid threshold range	[90, 140]
Correct rate (T=115)	$\approx$ 99.9998%
Minimum in valid range	> 99.7%
Meets $\geq$ 90% requirement	YES ✓